

Manual de Ingeniería: Bellman-Ford con Expansión de Estados

Este documento proporciona un análisis algorítmico exhaustivo de la versión avanzada y modificada del algoritmo de Bellman-Ford implementada en la clase `AlgoritmoBellmanFord`. Este código resuelve un problema clásico de grafos dirigidos (rutas simples en presencia de ciclos negativos) mediante la técnica de "Expansión de Estados".

1. Teoría Evolucionada del Algoritmo

El Problema Clásico de los Ciclos Negativos

En la teoría de grafos tradicional, el algoritmo de Bellman-Ford se utiliza para encontrar el camino más corto permitiendo pesos negativos. Sin embargo, si la red contiene un **ciclo negativo** (un bucle que reduce matemáticamente el coste total hacia menos infinito), el algoritmo estándar se ve obligado a abortar y devuelve un error. Físicamente, esto se debe a que el flujo (o el agua, en nuestro simulador) giraría infinitamente ganando energía ilimitada, lo cual es imposible.

La Solución Innovadora: Expansión de Estados

En lugar de bloquearse y fallar, la implementación actual resuelve el problema dividiendo la "realidad" del grafo en dos dimensiones o estados paralelos:

- Dimensión 1 (`rutas_simples`)**: Busca caminos estrictamente limpios. No permite *nunca* que se repita una estación.
- Dimensión 2 (`rutas_ciclo`)**: Si la Dimensión 1 detecta un ciclo negativo, abre una puerta a esta segunda dimensión. Aquí se permite continuar el viaje aprovechando la "energía extra" del bucle, pero bajo la estricta regla de que, a partir de ese momento, no se puede volver a repetir *ninguna* estación. Esto garantiza que el ciclo negativo se explota **exactamente una vez**.

2. Análisis Detallado de la Implementación en Python

2.1. Estructura de Memoria Expandida

```
# 1. INICIALIZACIÓN DE LA DOBLE MEMORIA
rutas_simples = {est: {'costo': float('inf'), 'camino': []} for est in
self.red.estaciones}
rutas_ciclo = {est: {'costo': float('inf'), 'camino': []} for est in
self.red.estaciones}
```

A diferencia del enfoque tradicional que solo guarda una matriz unidimensional de distancias `D[v]`, nuestro código almacena el **estado completo** de la ruta en cada nodo de forma proactiva. Al guardar la lista exacta `'camino'` dentro del diccionario, eliminamos la necesidad de la reconstrucción post-procesamiento con diccionarios de predecesores.

2.2. Validación Dinámica de Costos del Ciclo

```
def _calcular_costo_ciclo(self, ciclo_nodos, caudal_requerido):
    costo_total = 0
    # Iterar sobre los nodos del bucle para sumar los costos exactos
    # ...
    return costo_total
```

Cuando la línea de "rutas simples" detecta que un nodo está a punto de repetirse, no asume inmediatamente que es útil. El programa aísla el bucle interceptado, lo pasa por `_calcular_costo_ciclo` y suma el valor de sus tuberías. Solo si el balance total (`costo_ciclo < 0`) proporciona un beneficio real, se aprueba la transición a la Dimensión 2.

2.3. Bucle Principal y Reglas de Propagación

```
for _ in range(total_estaciones * 2): # ¡El doble de iteraciones!
for tuberia in self.red.tuberias:
    ...
    # --- A. INTENTAR EXPANDIR RUTAS SIMPLES ---
    if v not in rutas_simples[u]['camino']:
        # Propaga de forma normal
    else:
        # Si genera un ciclo negativo, se "teletransporta" a rutas_ciclo

    # --- B. INTENTAR EXPANDIR RUTAS DE CICLO ---
    if v not in rutas_ciclo[u]['camino']:
        # Se permite avanzar, pero con el "Escudo" cerrado para siempre
    ...
```

¿Por qué $total_estaciones * 2$ iteraciones?

En Bellman-Ford clásico, el número máximo de aristas de un camino simple es $V - 1$. Dado que nuestro algoritmo permite dar la vuelta por un ciclo completo antes de continuar, el camino máximo posible en el peor de los casos abarca casi todos los nodos dos veces. Duplicar las iteraciones externas garantiza matemáticamente la convergencia total de las rutas con bucle hacia el destino.

3. Análisis de Eficiencia (Complejidad Algorítmica)

3.1. Complejidad Temporal (Tiempo de Ejecución)

Analizaremos el peor caso absoluto, donde V es el número de vértices y E es el número de aristas.

- **Bucle principal:** Se repite exactamente $2 * V$ veces.
- **Bucle interno:** Itera sobre las E tuberías.
- **Operaciones internas:** La comprobación `v not in rutas_simples[u]['camino']` debe escanear una lista de tamaño máximo $2*V$, por lo que cuesta $O(V)$.
- **Evaluación de ciclos (El cuello de botella):** Si el programa intercepta bucles constantemente, la función `_calcular_costo_ciclo` recorre hasta V nodos, buscando entre E tuberías, tomando un tiempo $O(V * E)$.

Multiplicando estos factores de propagación continua, la complejidad asintótica en el peor de los casos hiper-conectados escala hasta $O(V^2 \cdot E^2)$. De forma amortizada (en topologías de tuberías normales donde no se desencadenan ciclos en cada iteración), se comporta cerca de $O(V^2 \cdot E)$.

3.2. Complejidad Espacial (Uso de Memoria)

El sacrificio por esta potencia de cálculo radica en el almacenamiento.

- En lugar de usar dos matrices planas de tamaño $O(V)$, almacenamos dos diccionarios completos.
- Cada una de las V estaciones guarda en su interior una matriz-camino (una lista de Python que puede crecer hasta $2V$ elementos).

Por lo tanto, la huella en la memoria RAM asciende a $O(V^2)$. Dado que las redes de aguas típicas (o modelados de grafos lógicos) no suelen superar las decenas de miles de nodos, este incremento espacial (unos pocos Megabytes en sistemas modernos) es un *Trade-off* (sacrificio de diseño de ingeniería) absolutamente justificado para poder resolver la intratabilidad de los ciclos negativos de manera perfecta y sin caídas.